

# Word Vectors

---

**Robin Jia & Xuezhe Ma**  
USC CSCI 544, Spring 2026  
January 20, 2026

# Announcements

---

- HW1 released on Brightspace
  - Deadline moved back to Monday, February 2 at 11:59pm
- Submit homework on Brightspace
  - We have decided to use Brightspace instead of Gradescope for homework submission

# “Will we learn about modern stuff?”

---

## YES

- Last class: Introducing classical approaches and showing that they don't yield deep language understanding
- This class: First building block we need for large language models
- Soon: Deep Learning & Transformers
- After that: Several new lectures on cutting-edge research and current practices in developing language models

# Logistic Regression Gradient

- Recall: We want to compute  $\arg \min_{w,b} \sum_{i=1}^n -\log \sigma(y^{(i)} \cdot (w^\top \phi(x^{(i)}) + b))$
- We minimize this with gradient descent
- What is the gradient?
- How? Apply  $\frac{d}{dx} \log \sigma(x) = \sigma(-x)$  and Chain Rule

Differentiate each  
term of sum  
independently

Loss function  $L(w, b)$

Chain Rule  
Deriv. of outside  
w.r.t inside

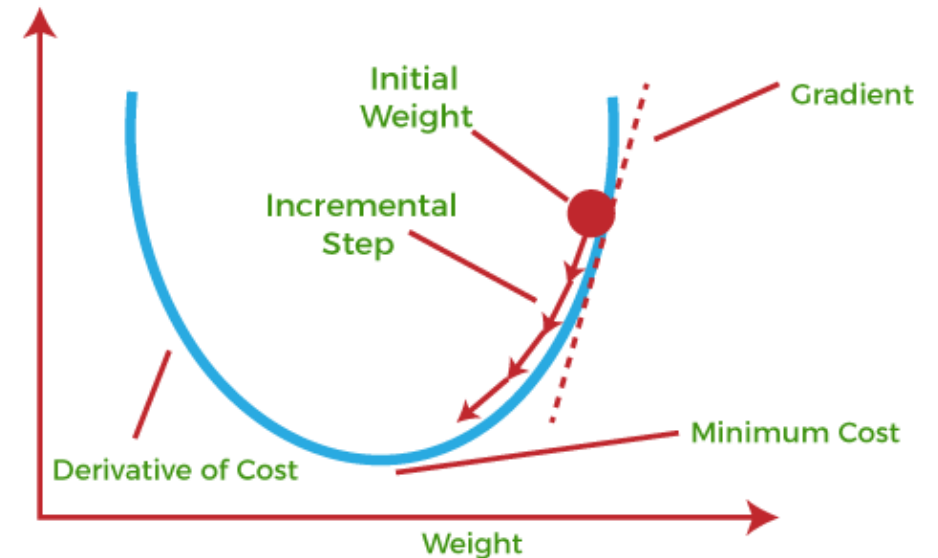
Chain Rule  
Deriv. of  
inside  
w.r.t.  $w$

$$\nabla_w L(w, b) = \sum_{i=1}^n \left[ -\sigma(-y^{(i)} \cdot (w^\top \phi(x^{(i)}) + b)) \right] \cdot \left[ y^{(i)} \cdot \phi(x^{(i)}) \right]$$

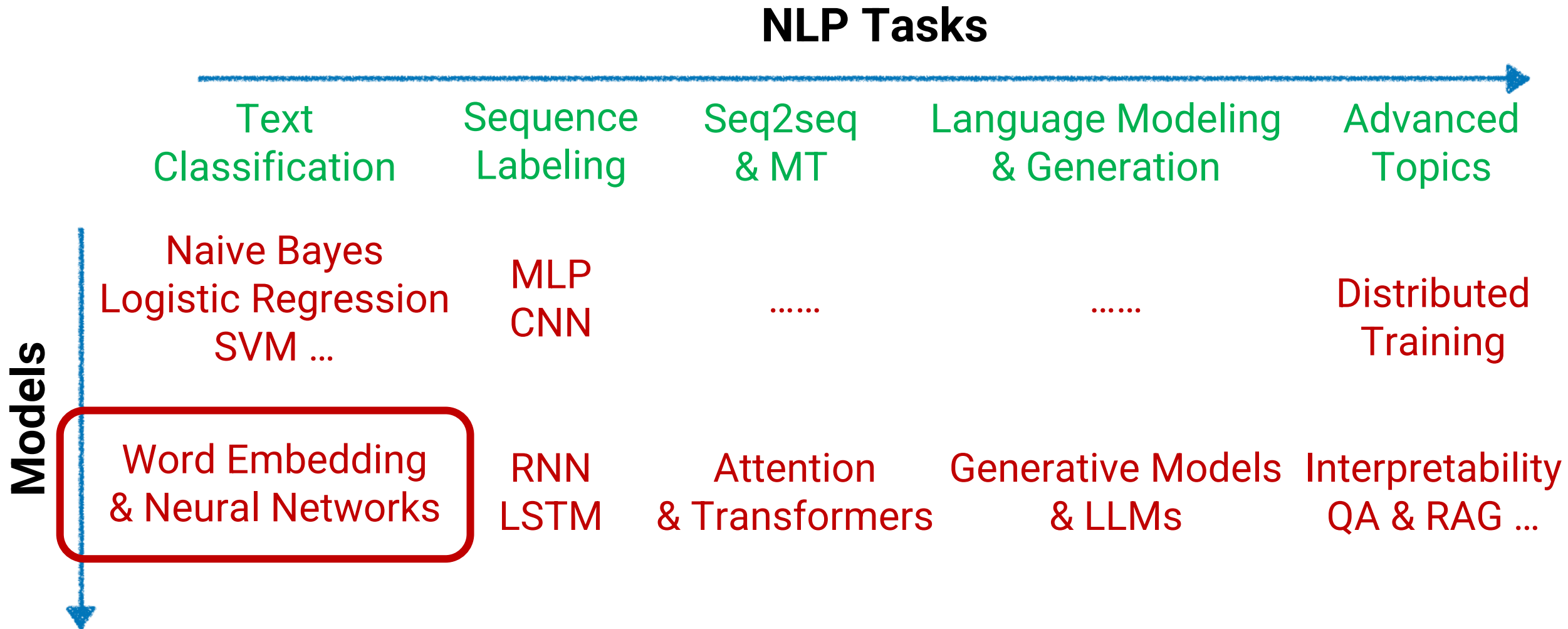
$$\nabla_b L(w, b) = \sum_{i=1}^n -\sigma(-y^{(i)} \cdot (w^\top \phi(x^{(i)}) + b)) \cdot y^{(i)}$$

# Logistic Regression & Convexity

- We run gradient descent to minimize the loss
- Is this guaranteed to work? Yes!
  - Logistic Regression loss function is **convex**
  - Intuitively: The graph of the function is “bowl-shaped”
  - Following gradient (= steepest descent direction) will get to the bottom of the bowl



# Course Overview



# Outline

---

- What are word vectors?
- Creating word vectors
  - Counting co-occurrences
  - Word2vec
- Using Word Vectors
  - Solving analogies
  - Bias in word vectors
  - As part of a classifier

# Word vectors

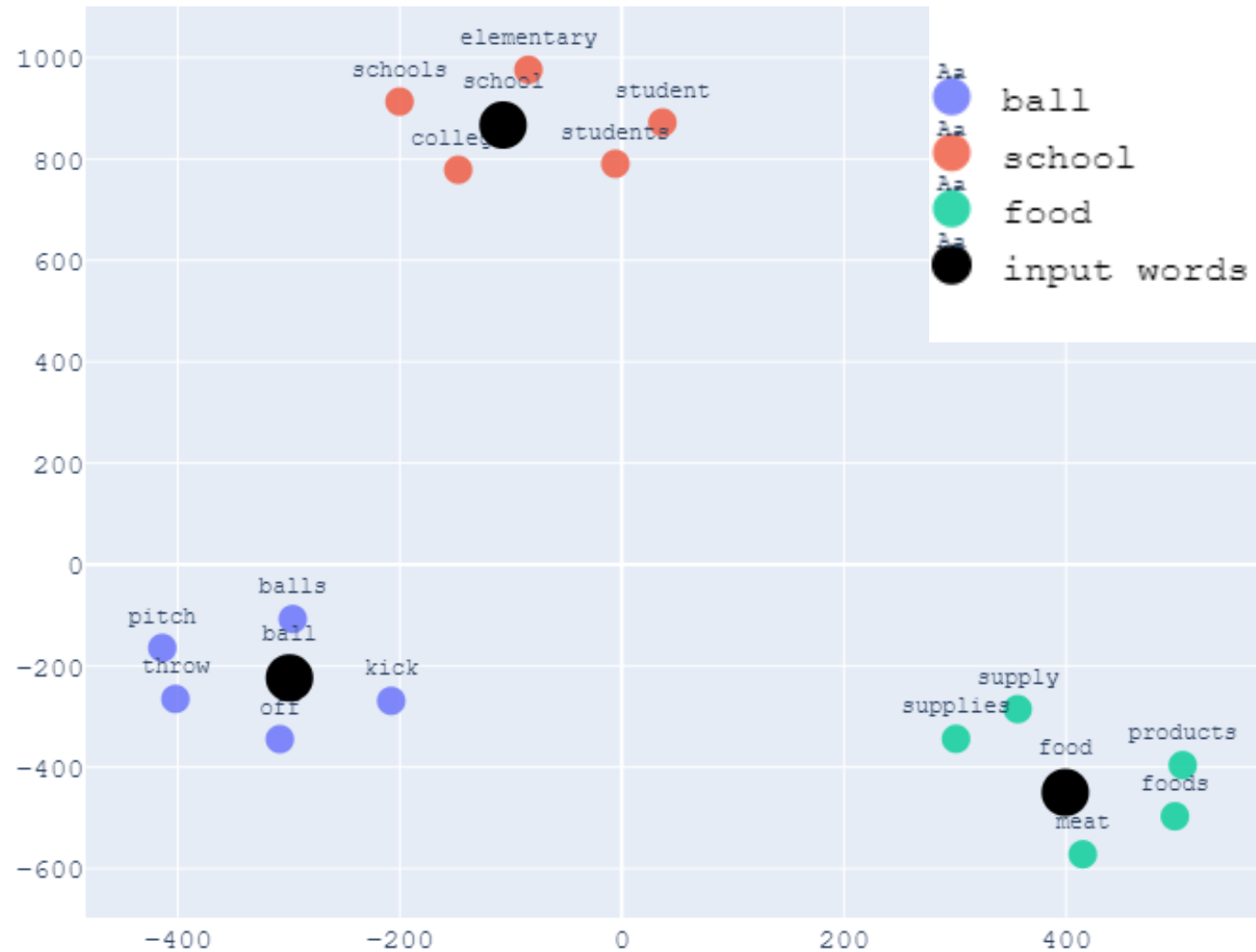
- Goal: For each word  $w$  in our vocabulary, learn vector  $v_w$  that represents word's meaning
  - Similar words should have similar vectors
  - Different components of the vector may represent different properties of a word
- Why?
  - Can be used as features for NLP models
  - Neural networks take vectors as inputs. To feed them sentences, need to represent each word as a vector

| Word $w$ | Vector $v_w$      |
|----------|-------------------|
| A        | [-0.4, 1.4, -1.2] |
| Aardvark | [2.2, -1.8, 0.6]  |
| Airport  | [0.7, 0.3, 3.1]   |
| ...      |                   |
| Elephant | [2.1, -1.3, 0.3]  |
| ...      |                   |
| Zoo      | [2.1, -1.4, 3.2]  |

Related to animals? Is a place

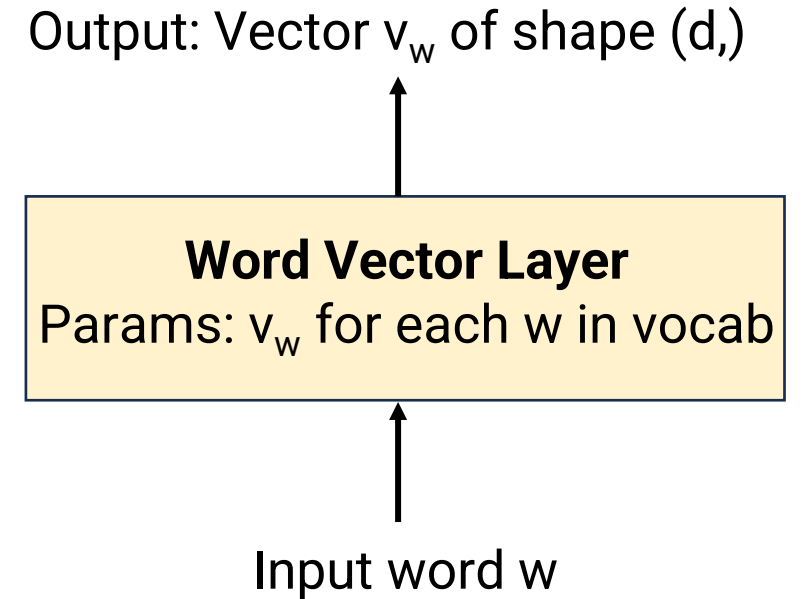
# Visualizing word vectors

- Goal: For each word  $w$ , learn vector  $v_w$  that represents word's meaning
  - Similar words should have similar vectors
  - Different components of the vector may represent different properties of a word



# A Word Vector “Layer” as a Function

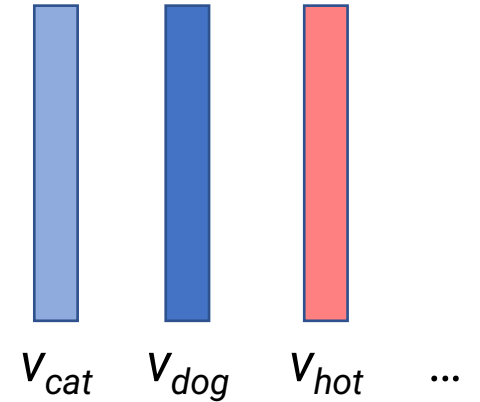
- Another view: A parametric function from vocabulary words to vectors
  - i.e., It has numerical parameters to learn
- Input  $w$ : A word (from our vocabulary)
- Output: A vector of length  $d$
- Formula: Return `word_vecs[w]`
- Parameters:
  - For each word  $w$  in vocabulary, there is a word vector parameter  $v_w$  of shape  $d$
  - $|V| * d$  total parameters needed
  - Think of this as a dictionary called `word_vecs`, where the keys are words & values are learned parameter vectors



# Lexical Semantics

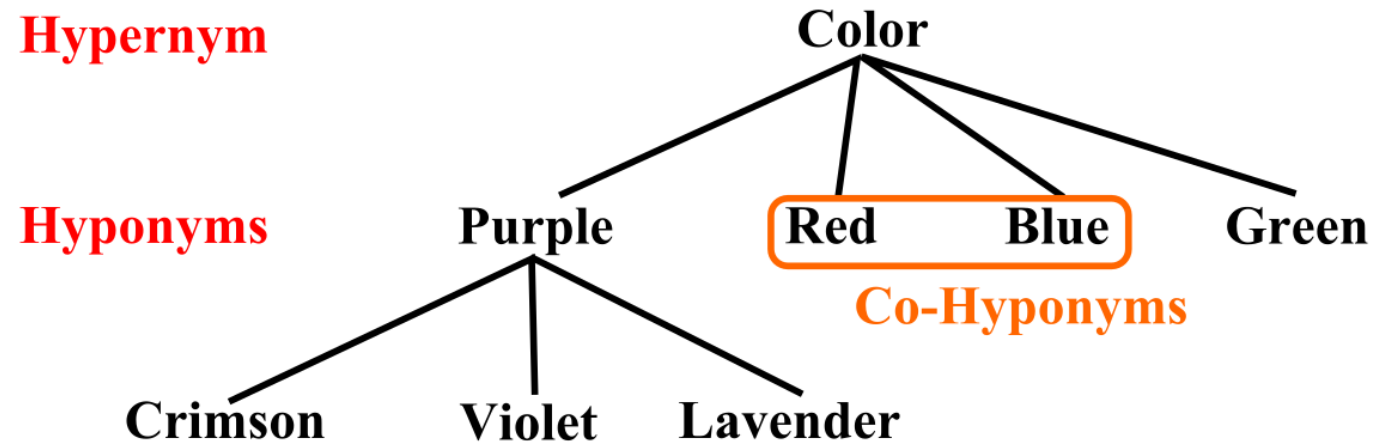
---

- Word vectors should capture *lexical semantics*
  - Lexical = word-level
  - Semantics = meaning
- What do we want to represent?
  - Synonymy (*car/automobile*) or antonymy (*cold/hot*)
  - Hypernymy/Hyponymy (*animal/dog*)
  - Similarity (*cat/dog, coffee/cup, waiter/menu*)
  - Various features
    - Sentiment (positive/negative)
    - Formality
    - All sorts of properties (Is a city? Is an action that a person can do?)



# Hypernyms and Hyponyms

- For a computer scientist
  - Hypernym = superclass
  - Hyponym = subclass
- Co-hyponyms are words that share the same hypernym



# The Distributional Hypothesis

---

- You hear a new word, **ongchoi**
  - ***Ongchoi** is delicious sauteed with garlic.*
  - ***Ongchoi** is superb over rice.*
  - *...**ongchoi** leaves with salty sauces...*
- Compare with similar contexts:
  - *...**spinach** sauteed with garlic over rice...*
  - *...**chard** stems and leaves are delicious...*
  - *...**collard greens** and other salty leafy greens*
- Conclusion: **ongchoi** is probably a leafy green similar to spinach, chard, and collard greens
- Distributional Hypothesis: Words appearing in similar contexts have similar meanings!
- Firth 1957: “You Shall Know a Word by the Company It Keeps”



# Outline

---

- What are word vectors?
- Creating word vectors
  - **Counting co-occurrences**
  - Word2vec
- Using Word Vectors
  - Solving analogies
  - Bias in word vectors
  - As part of a classifier

# Counting co-occurrences

- What is a word's "company"? Words that co-occur!
- Simple Idea: For each word, count how many times it co-occurs with every other word
  - Get a big corpus of text
  - Define co-occurrence in terms of a sliding window of words
  - Store all counts in a big table

... lemon, a tablespoon of apricot jam, a pinch...

Word w

Window of radius 2

```
counts["apricot"]["tablespoon"] += 1
counts["apricot"]["of"] += 1
counts["apricot"]["jam"] += 1
counts["apricot"]["a"] += 1
```

# Counting co-occurrences

- Result: Counts of each word with each possible neighbor word
  - “term-term” matrix
- Treat each row as the “word vector” for a word
- What is “ongchoi” most similar to?

| Word \ neighbors | wing | wheel | road | delicious | garlic |
|------------------|------|-------|------|-----------|--------|
| airplane         | 1000 | 200   | 40   | 10        | 0      |
| spinach          | 100  | 3     | 5    | 500       | 600    |
| truck            | 10   | 900   | 500  | 20        | 0      |
| apple            | 20   | 30    | 50   | 2000      | 20     |
| ongchoi          | 10   | 0     | 0    | 60        | 70     |

Co-occurs with “delicious” and “garlic” but not “wing”, “wheel”, and “road”

# Cosine similarity

- Given two vectors  $u$  &  $v$ , their **cosine similarity** is

$$\frac{u^\top v}{\|u\| \|v\|} = \cos(\theta)$$

where  $\theta$  = angle between the two vectors

- $\approx 1$ : Point in same direction
- $\approx 0$ : Orthogonal
- $\approx -1$ : Point in opposite direction

| Word \ neighbors | wing | wheel | road | delicious | garlic |
|------------------|------|-------|------|-----------|--------|
| airplane         | 1000 | 200   | 40   | 10        | 0      |
| spinach          | 100  | 3     | 5    | 500       | 600    |
| truck            | 10   | 900   | 500  | 20        | 0      |
| apple            | 20   | 30    | 50   | 2000      | 20     |
| ongchoi          | 10   | 0     | 0    | 60        | 70     |

`cossim("airplane", "ongchoi") = 0.11`

**`cossim("spinach", "ongchoi") = 0.99`**

`cossim("truck", "ongchoi") = 0.14`

`cossim("apple", "ongchoi") = 0.66`

**Conclusion: "ongchoi" is most similar to "spinach"**

# Dense vs. Sparse Vectors

---

- These vectors are:
  - Quite long: Do we need so many dimensions to capture lexical semantics?
  - Quite sparse: Many entries will be 0
- Can we find lower-dimensional dense vectors that capture word meaning?

# Outline

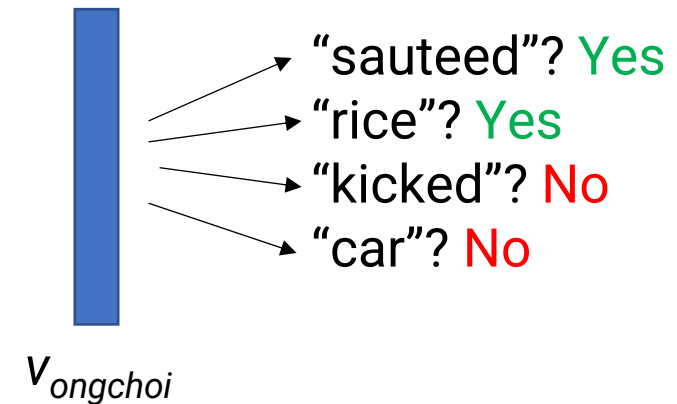
---

- What are word vectors?
- Creating word vectors
  - Counting co-occurrences
  - **Word2vec**
- Using Word Vectors
  - Solving analogies
  - Bias in word vectors
  - As part of a classifier

# Learning Dense Word Vectors

---

- Want to learn (low-dimensional) vector  $v_w$  for each word  $w$
- What makes a vector good?
- Idea:  $v_w$  should help you predict which words co-occur with  $w$ 
  - Captures **distribution** of context words for  $w$
  - Think of it as  $N$  binary classification problems, where  $N$  is size of vocabulary



# Creating a dataset

... lemon, a tablespoon of apricot jam a pinch...

Word  $w$

Window of radius 2

| Word $w$ ("input") | Context $w'$ ("task") | $y$ (label) |
|--------------------|-----------------------|-------------|
| apricot            | tablespoon            | +1          |
| apricot            | of                    | +1          |
| apricot            | jam                   | +1          |
| apricot            | a                     | +1          |

- Given: Raw dataset of text (unsupervised)
- We will create  $|V|$  "fake" supervised learning problems!
  - We don't really care about these supervised learning problems
  - We just care that we learn good vectors
- Task  $i$ : Did word  $w$  co-occur with the  $i$ -th word in the vocabulary?
  - Positive examples: Real co-occurrences within sliding window
  - Negative examples: Random samples

# Creating a dataset

... lemon, a tablespoon of apricot jam a pinch...

Word  $w$

Window of radius 2

| Word $w$ ("input") | Context $w'$ ("task") | $y$ (label) |
|--------------------|-----------------------|-------------|
| apricot            | tablespoon            | +1          |
| apricot            | of                    | +1          |
| apricot            | jam                   | +1          |
| apricot            | a                     | +1          |
| apricot            | seven                 | -1          |
| apricot            | forever               | -1          |
| apricot            | dear                  | -1          |
| apricot            | if                    | -1          |

- Given: Raw dataset of text (unsupervised)
- We will create  $|V|$  “fake” supervised learning problems!
  - We don’t really care about these supervised learning problems
  - We just care that we learn good vectors
- Task  $i$ : Did word  $w$  co-occur with the  $i$ -th word in the vocabulary?
  - Positive examples: Real co-occurrences within sliding window
  - Negative examples: Random samples

# How to sample negatives?

... lemon, a tablespoon of apricot jam, a pinch...

Word *w*

Window of radius 2

| Word <i>w</i> ("input") | Context <i>w'</i> ("task") | <i>y</i> (label) |
|-------------------------|----------------------------|------------------|
| apricot                 | tablespoon                 | +1               |
| apricot                 | of                         | +1               |
| apricot                 | jam                        | +1               |
| apricot                 | a                          | +1               |
| apricot                 | seven                      | -1               |
| apricot                 | forever                    | -1               |
| apricot                 | dear                       | -1               |
| apricot                 | if                         | -1               |

- Choose a fixed ratio of negative:positive (e.g. 2)
- Baseline: Sample according to frequency of word  $p(w)$  in the data
  - Not ideal because very common words ("the") get sampled a lot
- Improvement: Sample according to  $\alpha$ -weighted frequency

$$p_{\alpha}(w) = \frac{\text{count}(w)^{\alpha}}{\sum_{w' \in V} \text{count}(w')^{\alpha}}$$

- For  $\alpha < 1$ , high-frequency words get down-weighted
- Typically choose around  $\alpha=.75$

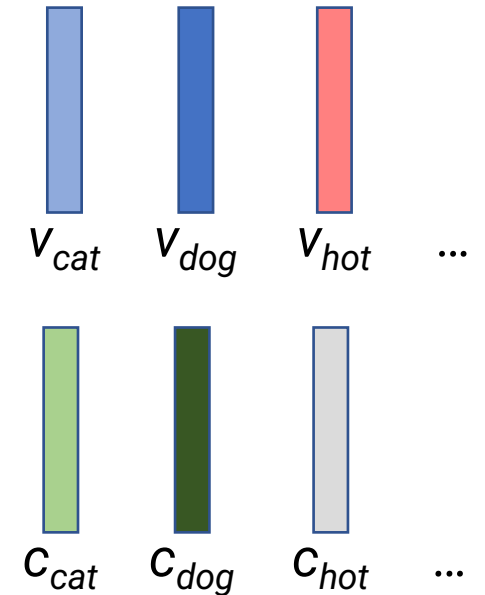
# word2vec model

- Parameters (all of dimension  $d$ ):
  - Word vector  $v_w$  for each word (“features”—the actual word vectors)
  - Context vector  $c_w$  for each word (“classifier weights” for task corresponding to  $w$  as context)
- Goal:  $v_w$  can be used by **linear classifier** to do **any** of the  $N$  “was this a context word” tasks
- Objective looks just like logistic regression:

$$L(v, c) = \sum_{(w, w', y)} -\log \sigma(y \cdot v_w^\top c_{w'})$$

Diagram illustrating the components of the logistic regression objective function:

- $(w, w', y)$ : The summation is over all word pairs  $(w, w')$  and their corresponding labels  $y$ .
- $w$ : word
- $w'$ : context
- $v_w$ : “features” for word
- $c_{w'}$ : “weight” for context



# Training word2vec

---

- Strategy: Gradient descent
- Gradient updates essentially same as logistic regression
  - Gradient w.r.t.  $c$  holds  $v$  fixed, so it's like  $v$  are fixed features

$$\nabla_{c_u} L(v, c) = \sum_{(w, w', y): w' = u} -\sigma(-y \cdot v_w^\top c_u) \cdot y \cdot v_w$$

Examples where  $w' = u$

Same as logistic regression  
where  $v_w$  is the input  $x$

- Gradient w.r.t.  $v$  is symmetrical

$$\nabla_{v_u} L(v, c) = \sum_{(w, w', y): w = u} -\sigma(-y \cdot v_u^\top c_{w'}) \cdot y \cdot c_{w'}$$

Examples where  $w = u$

Same as logistic regression  
where  $c_{w'}$  is the input  $x$

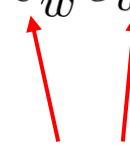
# Is this a convex problem?

---

- Looks a lot like logistic regression...
- But it's not convex!
- Why?
  - In logistic regression, we only optimize w.r.t. weights, features are constant
  - Now we optimize both at the same time!
- Fact to remember:  $f(x) = x_1 * x_2$  is not convex
- In practice, gradient descent still is quite effective

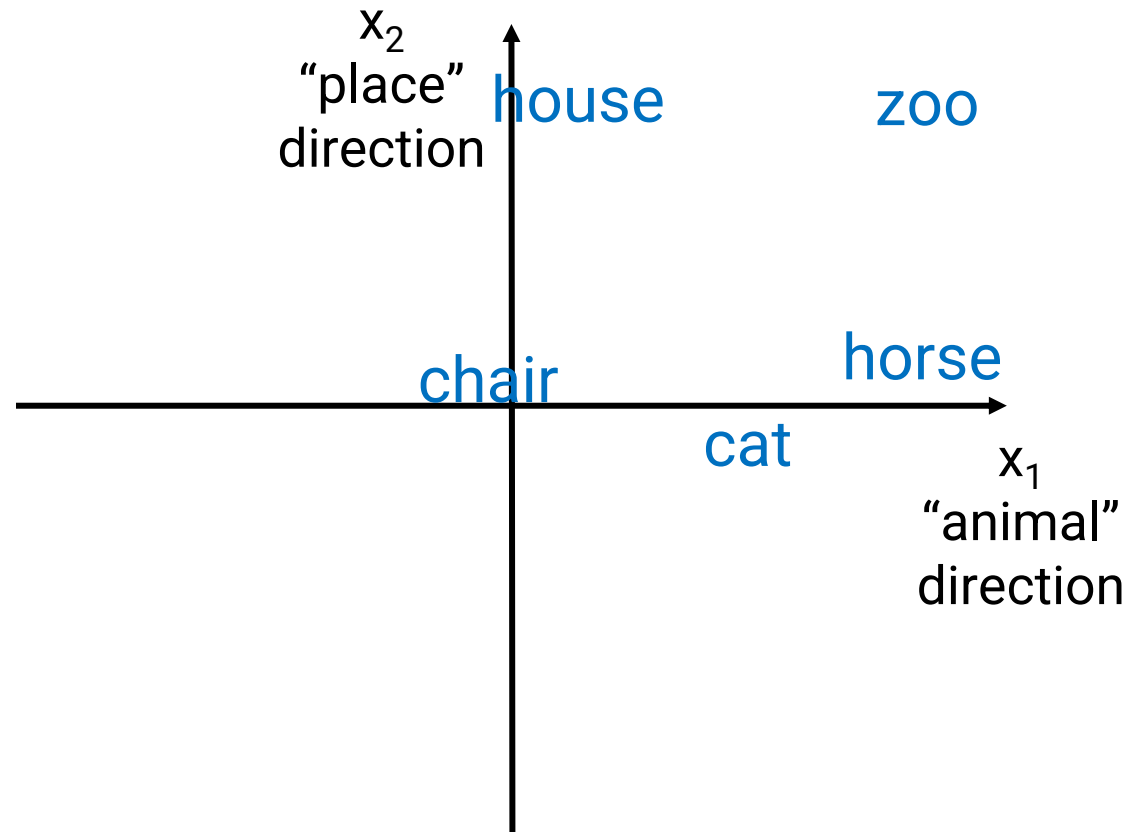
$$L(v, c) = \sum_{(w, w', y)} -\log \sigma(y \cdot v_w^\top c_{w'})$$

Both are optimization variables

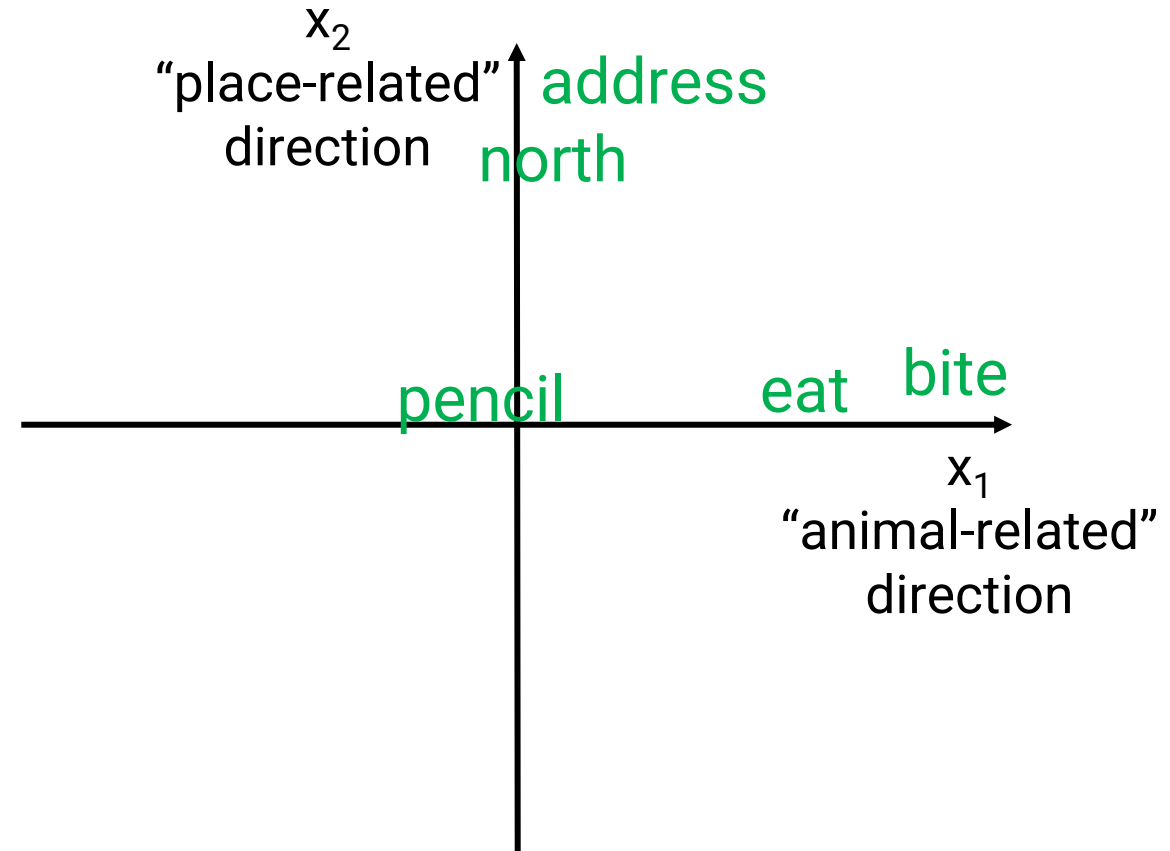


# Word vectors vs. Context vectors

## Word Vector Space

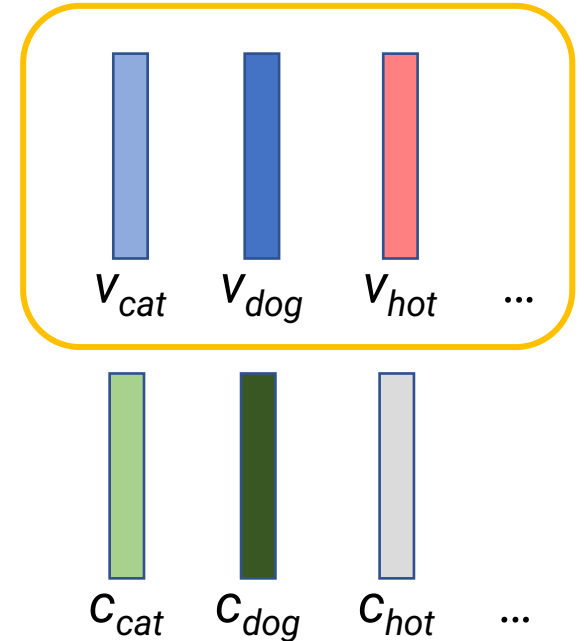


## Context Vector Space



# word2vec overview

- Acquire large unsupervised text corpus
- Create positive examples for every word by using sliding window
- Create negative examples by randomly sampling context word from weighted word frequency
- Train on logistic regression-like loss with gradient descent
- Return  $v$  vectors
  - $c$  vectors not needed—just helpers



# Outline

---

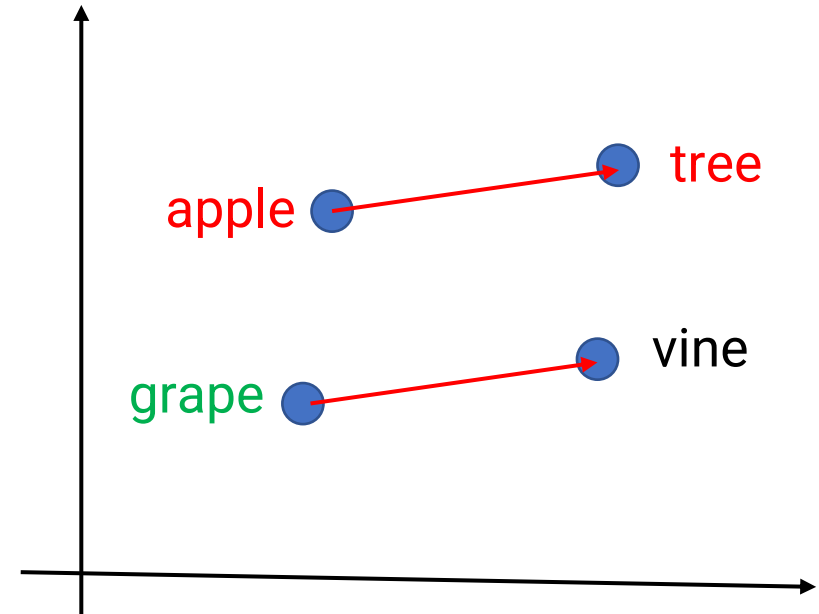
- What are word vectors?
- Creating word vectors
  - Counting co-occurrences
  - Word2vec
- Using Word Vectors
  - **Solving analogies**
  - Bias in word vectors
  - As part of a classifier

# Analogies in vector space

- *Apple is to tree as grape is to...*
- In vector space, resembles a parallelogram
  - Same relationship between apple and tree holds between grape and vine

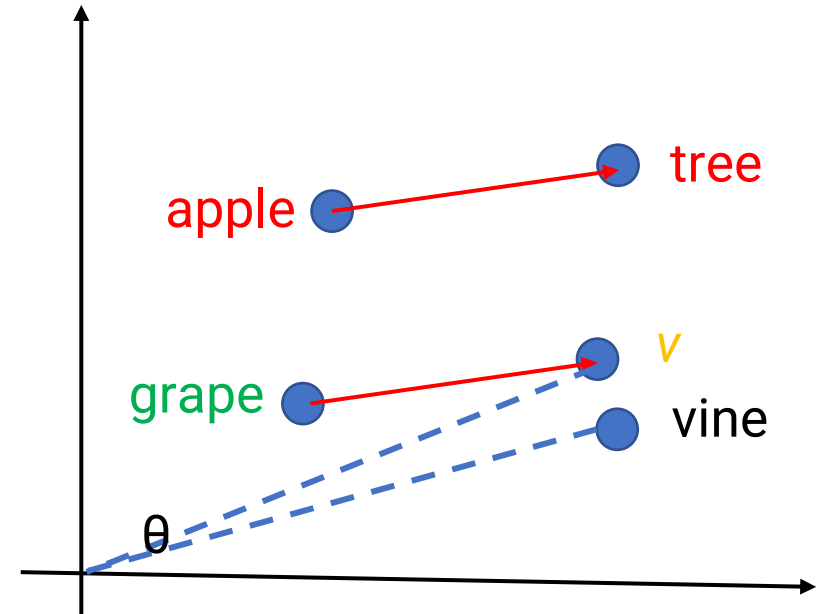
$$V_{vine} \approx \underbrace{V_{tree} - V_{apple}}_{\text{Represents the "grows on" relation}} + V_{grape}$$

Query word

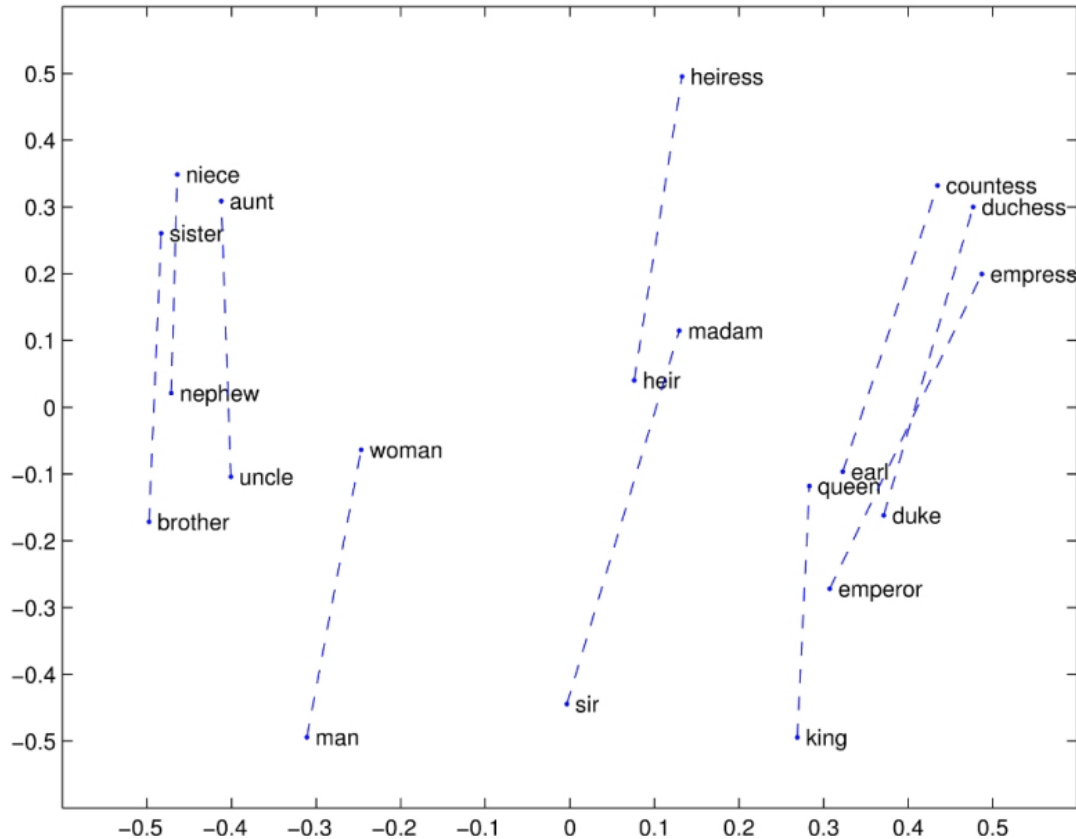


# Answering analogy queries

- Compute  $\mathbf{v} = \mathbf{V}_{tree} - \mathbf{V}_{apple} + \mathbf{V}_{grape}$
- Find word  $w$  in vocabulary whose  $\mathbf{V}_w$  is most similar to  $\mathbf{v}$ 
  - Common choice: Cosine similarity
  - Typically need to exclude words very similar to the query word (e.g. “*grapes*”)

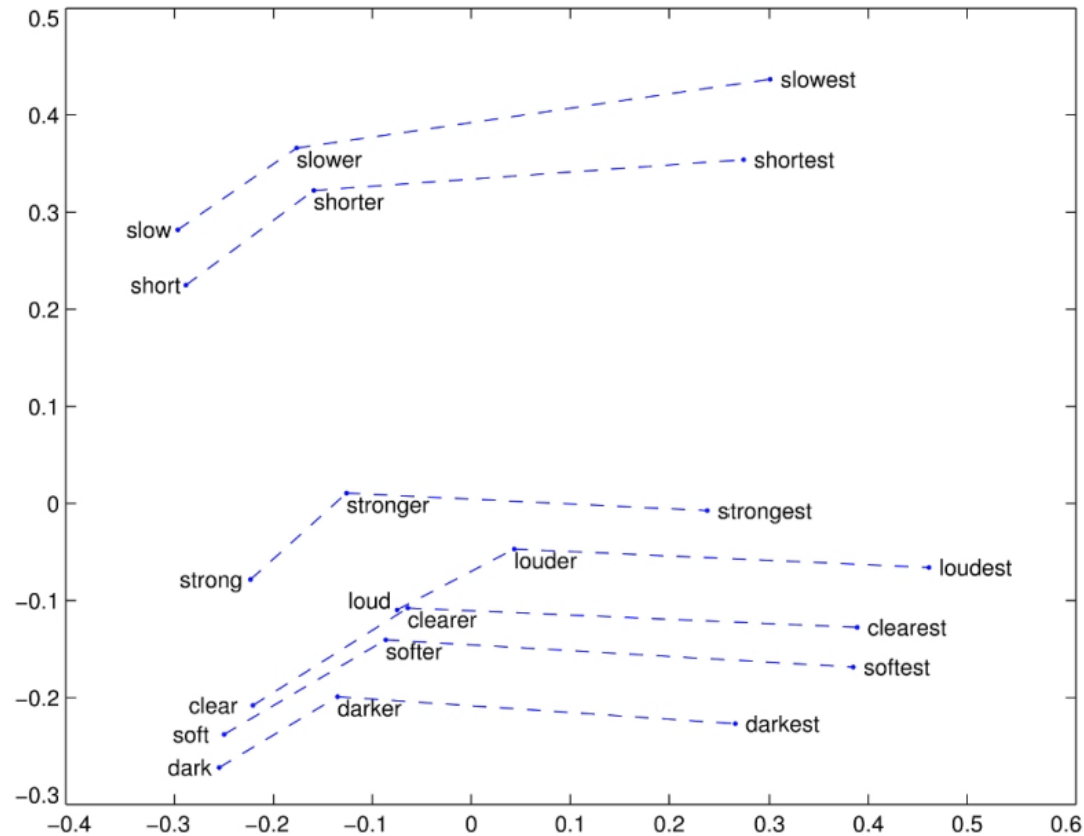


# Visualizing Analogies



- Figure: *Dimensionality reduction* to 2D, then plot words with known relationship
- Roughly same difference between male/female versions of the same word

# Visualizing Analogies



- Figure: *Dimensionality reduction* to 2D, then plot words with known relationship
- Roughly same difference between base, comparative, and superlative forms of adjectives

# Play around with word similarity

- [http://epsilon-it.utu.fi/wv\\_demo/](http://epsilon-it.utu.fi/wv_demo/)
- Select the English GoogleNews vectors

## Models

Select one of the available models

English GoogleNews Negative300 ▾

## Nearest words

Given a word, this demo shows a list of other words that are similar to it, i.e. nearby in the vector space.

**Case sensitive:** ☒ **Top N:**  ▾

melodious  
lilting  
sonorous  
lilt  
melody  
plaintive  
timbre  
baritone  
staccato  
soulful

# Semantle

- <https://semantle.com/>

## SEMANTLE 🔍



The nearest word has a similarity of **67.56**, the tenth-nearest has a similarity of **43.9** and the thousandth nearest word has a similarity of **28.97**

Game #1451

Guess

Hint

Give Up

Play Junior

Play Archive

Teams

## FAQ

### How to play?

The objective is to guess the secret word.

Each guess must be a single word. Semantle will inform you how semantically similar your guess is to the secret word.

Unlike other word games, this game is not about spelling; it's about meaning. We calculate this meaning using artificial intelligence (specifically word2vec technology).

In word2vec, each word has a measurable semantic distance from another, indicating their level of relatedness. Once you get within one thousand words of the secret word, we will tell you in the proximity column.

You have unlimited guesses! Good luck!

Finding it too hard? Try [Semantle Junior](#)

# Outline

---

- What are word vectors?
- Creating word vectors
  - Counting co-occurrences
  - Word2vec
- Using Word Vectors
  - Solving analogies
  - **Bias in word vectors**
  - As part of a classifier

# Machine learning is a tornado

---

- ...it picks up everything in its path
- Data has all sorts of associations we may not want to model

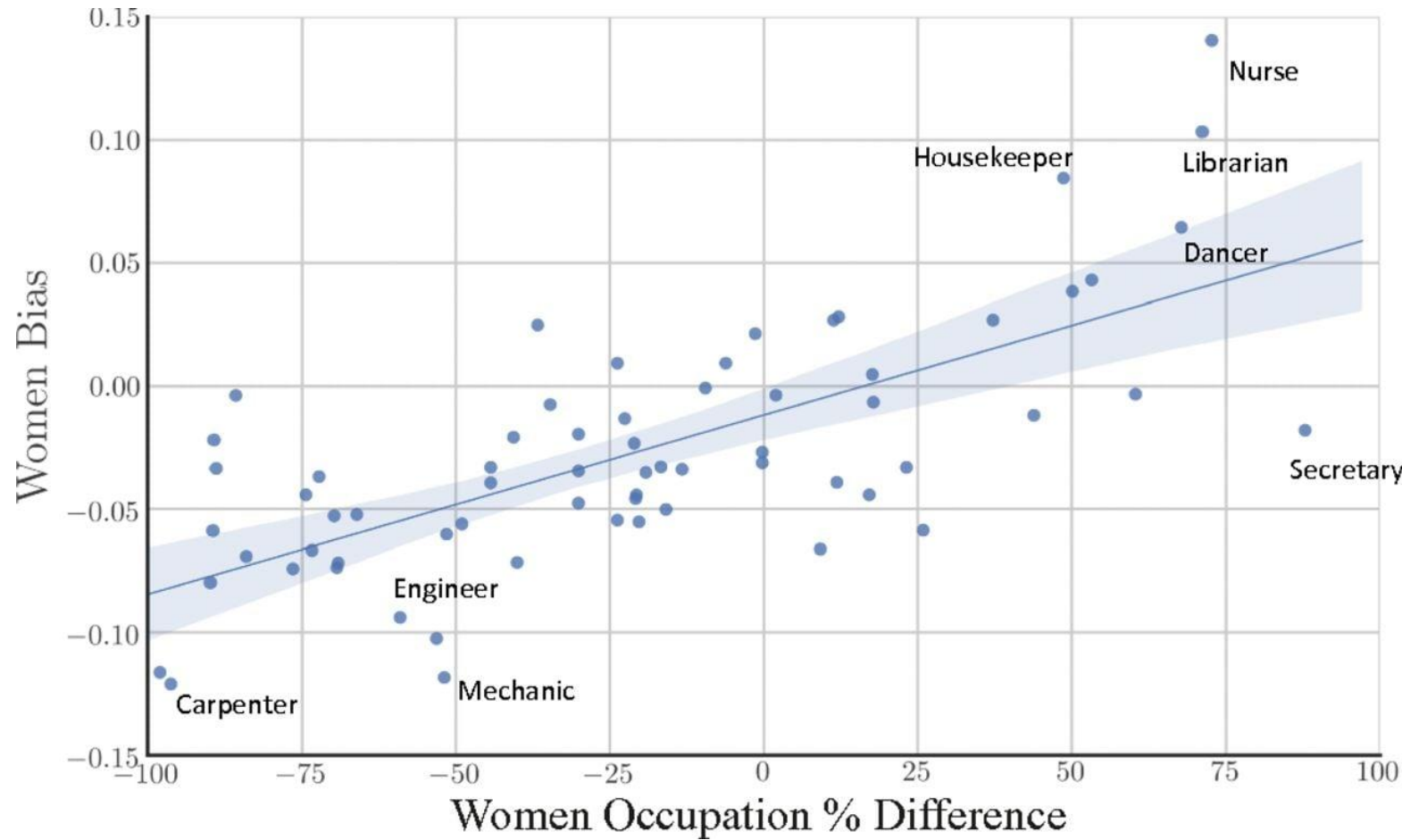


# What word associations are out there?

---

- What is *programmer* – *man* + *woman*?
  - According to word vectors trained on news data, it's *homemaker*
  - Existing data has tons of correlations between occupation and gender
- word2vec doesn't know what is a semantic relationship and what is a historical correlation
  - “*queen*” is more related to “*she*” than “*he*” semantically
  - “*nurse*” may co-occur more with “*she*” than “*he*” in available data but not a semantic relationship!

# Word vectors quantify gender stereotypes



- X-axis: Real percentage difference in workforce between women & men
- Y-axis: Embedding bias = difference of distance from male-related words and female-related words
- Strong correlation!

# Outline

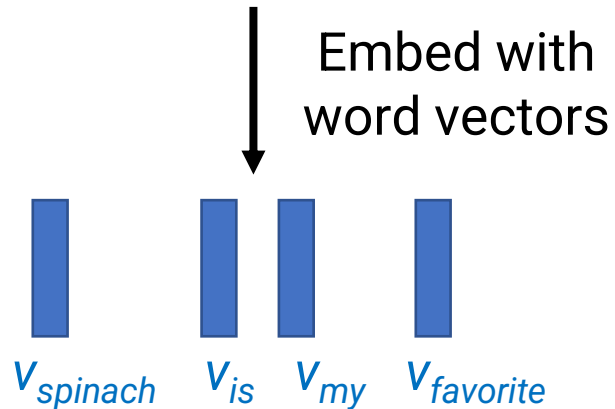
---

- What are word vectors?
- Creating word vectors
  - Counting co-occurrences
  - Word2vec
- Using Word Vectors
  - Solving analogies
  - Bias in word vectors
  - **As part of a classifier**

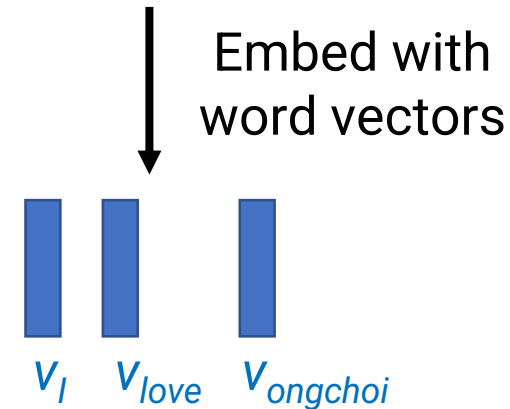
# Word Vectors and Classification

- Can we use word vectors for text classification?

*Spinach is my favorite*

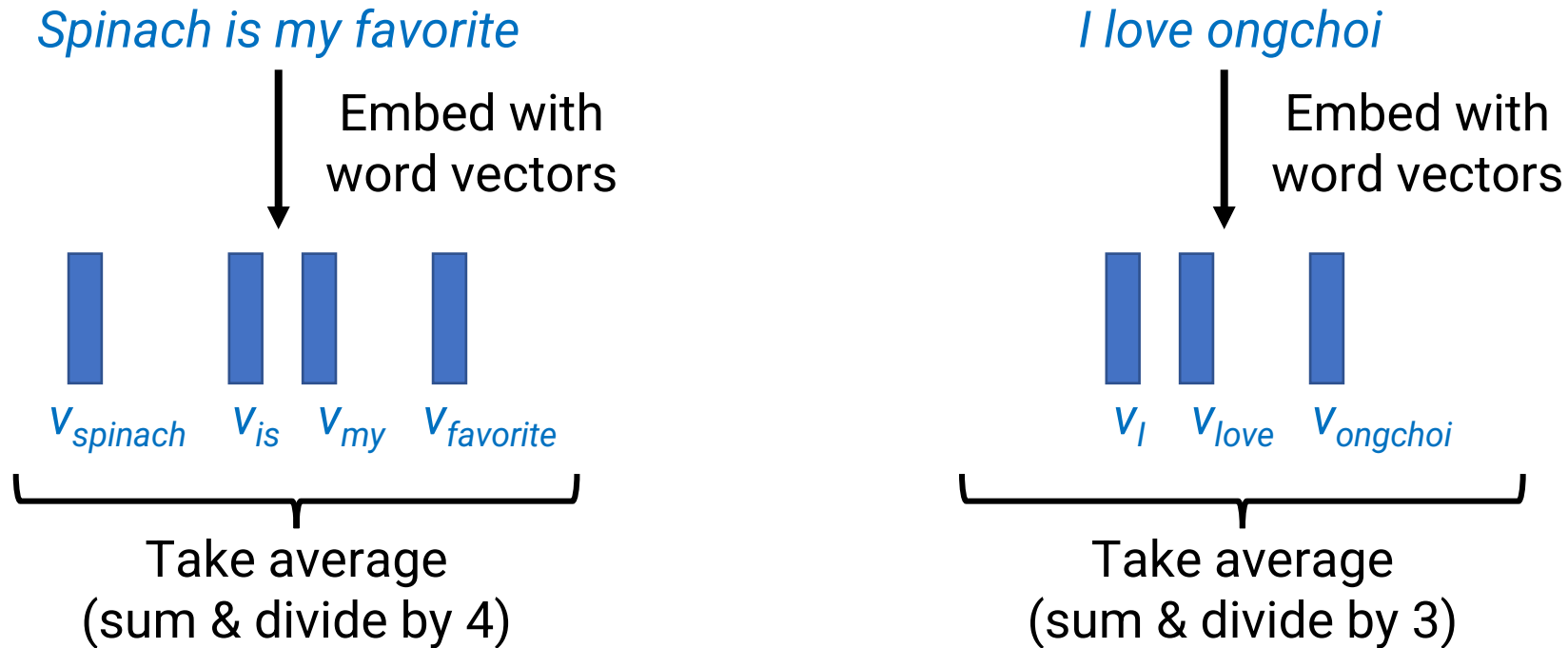


*I love ongchoi*



- Classifier expects a fixed-size feature vector, learns fixed number of weights
- How do we deal with variable number of words per document?

# A Simple Approach: Mean Pooling



- Simplest approach: Mean pool = simple average
- Final result is same dimension as a single word vector
- Note: Each word in shorter sentence now implicitly has higher "importance"

# Why do Word Vectors Help?

---

- Generalization to words not seen in training data
- Key idea: We learn something about language from a large, general corpus before we train a classifier for our specific problem
- Scenario: A rare word (e.g., “mellifluous”) appears in a test document that was never seen in training data
  - Using unigram count features:
    - No way to learn what this word means
    - Model has to just ignore this word
  - Using word vector features:
    - Word vector for rare word may be similar to a more common word (e.g., “soulful”) seen during training
    - Model can treat this word like it was this more common word

# Remaining Issues

---

- Still a bag-of-words model (no word order, phrase information)
- Has some lexical semantics, but what about *polysemy*?
  - Polysemy: Words can have multiple possible meanings depending on context

*“plant”*



*“bank”*



*“bat”*



- *Word sense disambiguation*: Classic task of determining which “sense” of a word is being used in a given sentence
  - Requires looking at the broader sentence-level context

# Word vectors and polysemy

For word vectors trained on **Google News**, what are the nearest neighbors for...

*“plant”*

Show nearest

Case sensitive: ☒

Top N:

plants  
Plant  
factory  
factories  
refinery  
Plants  
mill  
facility  
smelter  
manufacturing

**Factory**

*“bank”*

Show nearest

Case sensitive: ☒

Top N:

banks  
banking  
Bank  
lender  
banker  
depositors  
mortgage\_lender  
depositor  
BofA  
Citibank

**Financial institution**

*“bat”*

Show nearest

Case sensitive: ☒

Top N:

bats  
batting  
batsman  
batted  
Batting  
batsmen  
fielder  
wicket\_keeper  
seamer  
fielders

**Sports (cricket!)**

**Source of data really matters!**

# How to Improve?

---

- Need to model **interactions between words**
  - E.g., “plant” means something different in context of manufacturing
- Need to model **order of words**
  - E.g., “The cat saw the mouse” vs. “The mouse saw the cat”

# Conclusion

- Distributional hypothesis: Words that appear in similar contexts have similar meanings
- word2vec: Learn vectors by inventing a prediction problem (did this word-context pair really occur in the text?)
- Vector arithmetic lets us complete relations
- Vectors capture both lexical semantics and historical biases
- Next time: Word vectors as a component of neural networks for processing text

Word  $w$

... lemon, a tablespoon of apricot jam, a pinch...

Window of radius 2

